Studio Nx

JS 273 — 장치 전체 채우기와 이력 알림

2026-10-19

JavaScript 273 — 장치 전체 채우기와 이력 알림 (`sequencer-devices.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-10 프로젝트 전체 저장·불러오기: applyFullProject · setDeviceNotifier · 이력 도우미 (PRD 02 FR-254~255)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	272 시퀀서 시작과 전체 불러오기 · (다음 작업) 클립 · 마커 저장

1. 하는 일

`/full` 로 받은 데이터로 트랙과 장치를 화면에 그린 뒤, 장치를 바꿀 때마다 `setDeviceNotifier` 로 연결한 곳(`sequencer.js` `handleChange`)에 알립니다.

```
applyFullProject(data)
 ├ tracks 배열 돌면서 renderTrack
 ├ instruments / effects 배열 돌면서 ensureDeviceRow
 ├ restoreOpenPanels — 열려 있던 칸 다시 열기
 └ open 칸이면 ensureLiveCells — 실시간 뷰 셀 채우기
        ↓
장치 추가 · 고치기 · 떼기 · 순서 바꾸기
        ↓
notify({ type: 'deviceAdd' | 'deviceUpdate' | 'deviceRemove' | 'deviceMove', … })
        ↓
sequencer.js handleChange → pushHistory + markDirty
```

2. 언제 쓰나

쓰는 곳	
`sequencer.js` `loadProject`	`applyFullProject(data)` 를 한 번 부릅니다
`sequencer.js` `init`	`setDeviceNotifier(handleChange)` 로 알림을 연결합니다
`moveEffect` · `addDevice` …	성공한 뒤 `notify(…)` 를 부릅니다
되돌리기 도우미	`reapplyDevice` · `dropDevice` · `restoreDevice` · `moveEffectSlots` 는 undo/redo 가 바로 부릅니다

3. 코드

```
let notify = function () {};

export function setDeviceNotifier(fn) {
  notify = typeof fn === 'function' ? fn : function () {};
```

```
}

export function applyFullProject(data) {
  const tracks = (data && data.tracks) || [];
  document.getElementById('tracks').innerHTML = '';
  tracks.forEach(function (track) {
    renderTrack(track);
    (track.instruments || []).forEach(function (inst) { ensureDeviceRow(track, 'instrument', inst); });
    (track.effects || []).forEach(function (fx) { ensureDeviceRow(track, 'effect', fx); });
  });
  restoreOpenPanels();
}

export function reapplyDevice(trackId, kind, deviceId, patch) {
  const track = trackById(trackId);
  const device = deviceById(track, kind, deviceId);
  if (!track || !device) return false;
  updateDevice(track, kind, device, patch, { silent: true });
  return true;
}

export function dropDevice(trackId, kind, deviceId) {
  const track = trackById(trackId);
  const device = deviceById(track, kind, deviceId);
  if (!track || !device) return false;
  removeDevice(track, kind, device, { silent: true });
  return true;
}

export function restoreDevice(trackId, kind, snapshot) {
  const track = trackById(trackId);
  if (!track) return false;
  const restored = addDevice(track, kind, snapshot, { silent: true });
  return Boolean(restored);
}

export function moveEffectSlots(trackId, from, to) {
  const track = trackById(trackId);
  if (!track) return false;
  moveEffect(track, from, to, { silent: true });
  return true;
}
```

문법 풀이

식	뜻
`document.getElementById('tracks').innerHTML = ''`	이전 화면을 비운 뒤 서버 순서대로 다시 그립니다
`notify({ type: 'deviceAdd', … })`	장치가 바뀐 사실을 시퀀서 시작점에 알립니다
`{ silent: true }`	되돌리기 도우미가 다시 `notify` 를 부르지 않도록 막습니다
`deviceById(track, kind, deviceId)`	트랙 안에서 장치를 찾습니다. 없으면 null
`Boolean(restored)`	addDevice 가 뭘 돌려주든 항상 true/false 로 맞춥니다
`trackById` export	되돌리기 도우미와 시험에서 트랙을 찾는 데 사용합니다

4. 왜 이렇게

선택	이유
`applyFullProject` 가 한 번에 다시 그림	부분 갱신은 코드가 복잡해지고, 서버가 이미 전부 줬으므로 그림을 맞추는 데 안전합니다
`notify` 는 모듈 안에 하나	`setDeviceNotifier` 로 시켜서 시작점이 연결합니다. 장치 모듈이 이력 모듈을 직접 알 필요가 없습니다
되돌리기 도우미를 export	undo/redo 가 같은 경로로 장치를 되돌립니다 (새로운 코드를 만들지 않음)
`silent` 플래그	되돌리기 중에 다시 이력이 쌓이면 어긋납니다. `history.js` `applying` 과 짝을 이룹니다
헤더 주석의 FR-2197 · 클립 불러오기는 유보	프런트에서 연결선 저장과 클립 불러오기는 이번에 넣지 않았습니다. 사용자 검토 때 다음 작업으로 제안합니다

5. 시험

`docs/handoff/s1-10_tests/ui.mjs` 에서 `/full` 후 `#tracks` 아래 트랙 줄과 `data-device-id` 칸이 들어갔는지, 이펙트 순서를 바꾼 뒤 Ctrl+Z 로 이전 순서로 돌아오는지, 장치를 떼고 Ctrl+Z 하면 다시 돌아오는지, 되돌리기 중에 새 pushHistory 가 생기지 않는지 확인합니다.